

Evolução do SteamTwo

Biblioteca pessoal, comparador de jogos e integração preparada para PostgreSQL

GRUPO	Grupo Resident Corinthians
INTEGRANTES	Guilherme Nomoto Matuhashi José Octávio Zansavio Pereira
REPOSITÓRIO	https://github.com/ souvlagyi/steamtwo-grupo-resident-corinthians

FATEC - Análise e Desenvolvimento de Sistemas - 2026

1. O que foi entregue

Esta versão do SteamTwo parte do projeto-base indicado na atividade e amplia a experiência do catálogo com recursos autorais orientados à decisão do usuário: uma biblioteca pessoal com progresso e notas, um comparador de dois ou três jogos e uma rotina de dados locais para demonstrar a integração com PostgreSQL.

Projeto-base	https://github.com/DavidSilvaProg/steamtwo
Novo repositório	https://github.com/souvlagyi/steamtwo-grupo-resident-corinthians
Tecnologias	React + Vite no front-end; Node.js + Express na API; PostgreSQL e biblioteca pg na persistência.
Segurança	O repositório possui somente .env.example. O arquivo .env com a URL do banco e eventuais chaves não deve ser publicado.

Como a entrega atende à atividade

Critério	Entrega do grupo	Evidência
Back-end e PostgreSQL	Migrações de domínio e da biblioteca; repositórios SQL separados; seed local.	Migrations 001 e 002; rotas /api/library; comando db:seed.
Melhorias relevantes	Biblioteca com status/nota e comparador de jogos com resumo de diferença e gêneros comuns.	Interface, endpoints e testes específicos.
Integração e qualidade	React consome API Express; serviços isolam regra; validações Zod; foco visível e estado ARIA.	26 testes automatizados e build validado.
Documentação	README, instruções, registro de testes e este PDF com capturas.	Pasta docs/ no novo repositório.

2. Melhorias autorais

Melhoria	Como funciona	Benefício para o usuário
Minha biblioteca	Salva o jogo no perfil local com os status Quero jogar, Jogando ou Concluído e uma nota de até 280 caracteres.	Organiza a próxima experiência de jogo e mantém o contexto do usuário.
Comparador	Permite escolher até três títulos e comparar índice, popularidade histórica, tendência, lojas e gêneros.	Ajuda a decidir entre jogos sem alternar entre várias telas.
Seed do banco	Insere catálogo e rankings de demonstração após as migrações, sem exigir credenciais externas.	Permite comprovar interface, API e banco localmente.
Validação e acessibilidade	Zod valida slugs, status e quantidade de itens; botões de comparação expõem aria-pressed e foco visível.	Evita requisições inválidas e torna o fluxo mais claro para teclado/leitor de tela.

Endpoints acrescentados

Método	Endpoint	Resposta esperada
GET	/api/compare?slugs=jogo-a,jogo-b	Dois ou três jogos e um resumo com diferença de índice e gêneros compartilhados.
GET	/api/library	Itens salvos com status, nota e dados do jogo.
PUT	/api/library/:slug	Inclui ou atualiza status e nota do jogo selecionado.
DELETE	/api/library/:slug	Remove o jogo da biblioteca do perfil local.

A implementação evita replicar os campos do catálogo na biblioteca: a tabela `game_library` guarda apenas a referência ao jogo, estado e nota. Ao listar a biblioteca, o repositório SQL monta uma visão com gêneros, lojas e indicadores do catálogo.

3. Integração com PostgreSQL

A interface React envia requisições HTTP para a API Express. As rotas validam entradas com Zod e chamam serviços de catálogo ou biblioteca. Com DATABASE_URL configurada, esses serviços usam repositórios SQL baseados em pg e consultas parametrizadas. A biblioteca é persistida em game_library, ligada ao catálogo pela chave game_id.

Interface React Catálogo, biblioteca e comparação	API Express Rotas e validação	Serviços Regras de domínio	Repositórios SQL pg + parâmetros	PostgreSQL Tabelas migradas
---	----------------------------------	-------------------------------	-------------------------------------	--------------------------------

Migrações e dados de demonstração

Arquivo/comando	Papel
001_create_steamtwo_domain	Cria jogos, gêneros, lojas, snapshots e rankings.
002_create_game_library	Cria game_library com status, nota limitada a 280 caracteres e chave estrangeira para games.
npm run db:migrate	Aplica o esquema na instância PostgreSQL configurada.
npm run db:seed	Inclui jogos e rankings de demonstração para que as telas funcionem com dados do banco.
GET /api/health	Executa SELECT 1 quando DATABASE_URL está configurada e retorna database.status: connected.

Antes da submissão, execute db:migrate e db:seed usando o DATABASE_URL do grupo e inclua no PDF a captura de /api/health com database.status igual a connected. A máquina usada para preparar este modelo possuía o serviço PostgreSQL, mas não tinha a senha da instância disponibilizada.

4. Evidências de funcionamento

As capturas abaixo foram registradas localmente após iniciar a API. Elas demonstram a navegação e as funcionalidades do grupo em modo de demonstração. Com o banco do grupo configurado e populado, as mesmas telas passam a receber os dados pelos repositórios PostgreSQL.

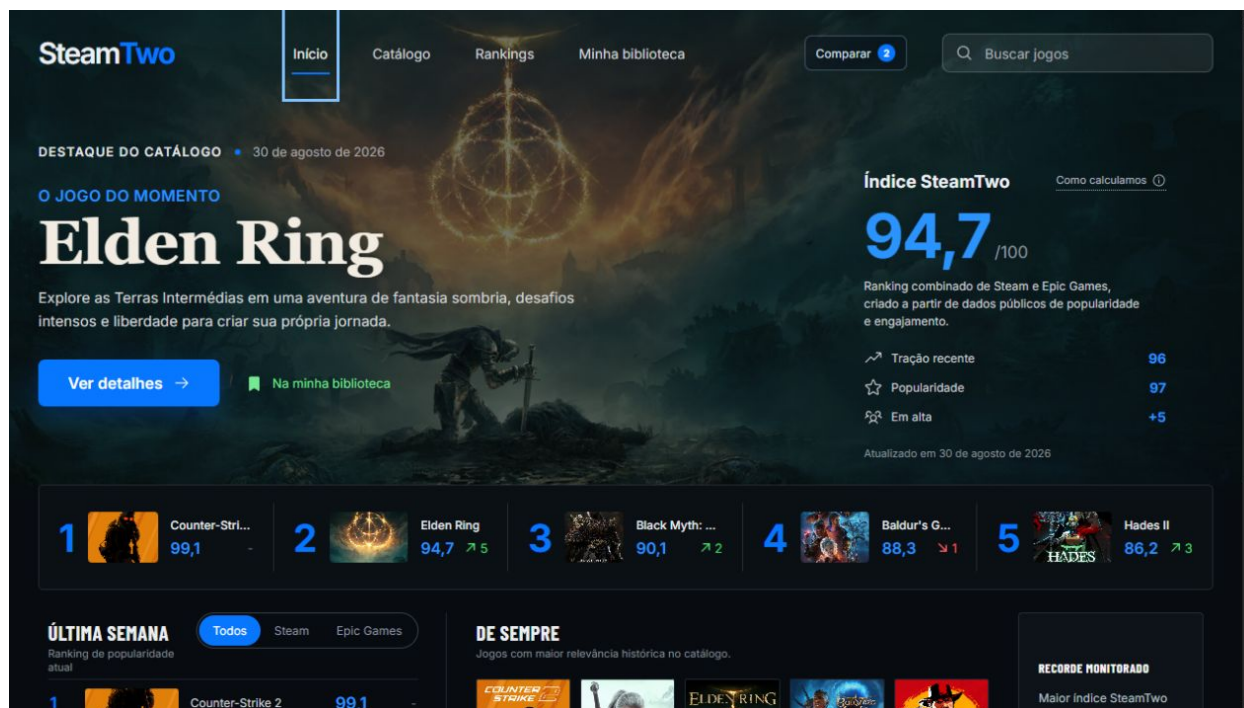


Figura 1 - Dashboard consumindo a API do SteamTwo e exibindo o destaque, ranking e indicadores.

Na figura 1, Elden Ring foi adicionado à biblioteca e dois jogos já estão selecionados para comparação. O indicador no cabeçalho acompanha a seleção do usuário.

5. Biblioteca e comparação

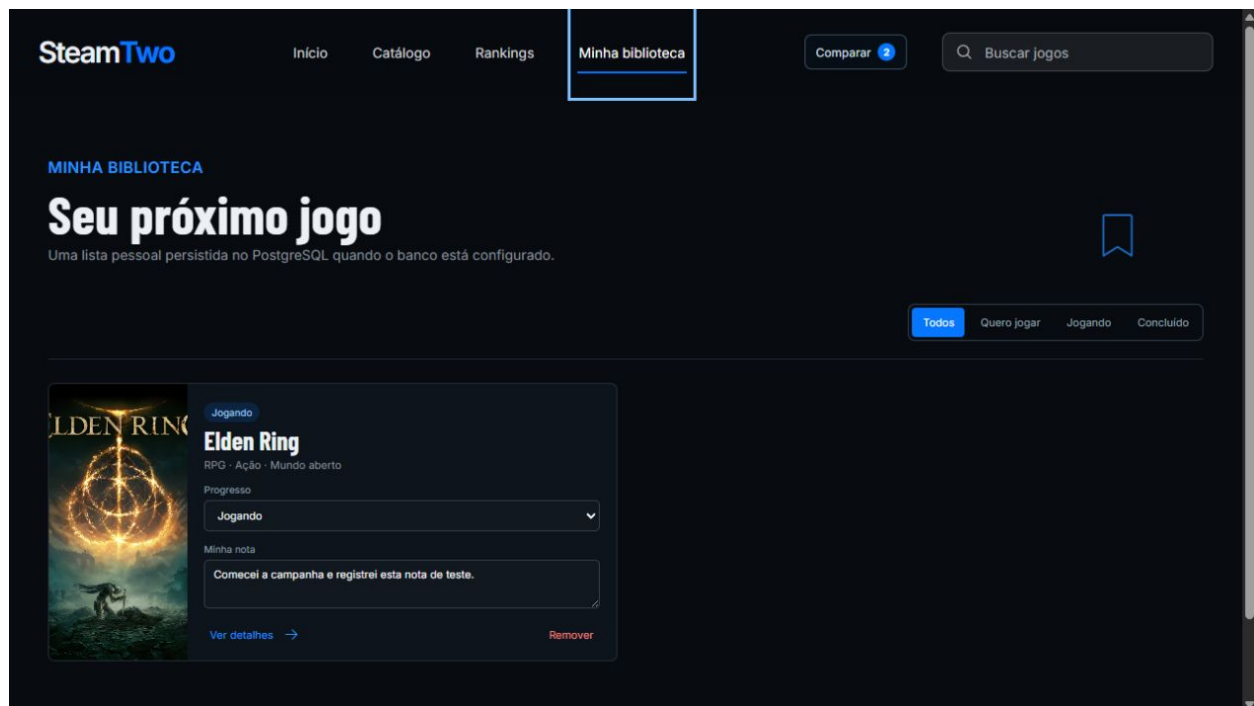


Figura 2 - Tela Minha biblioteca com Elden Ring, status Jogando e nota registrada.

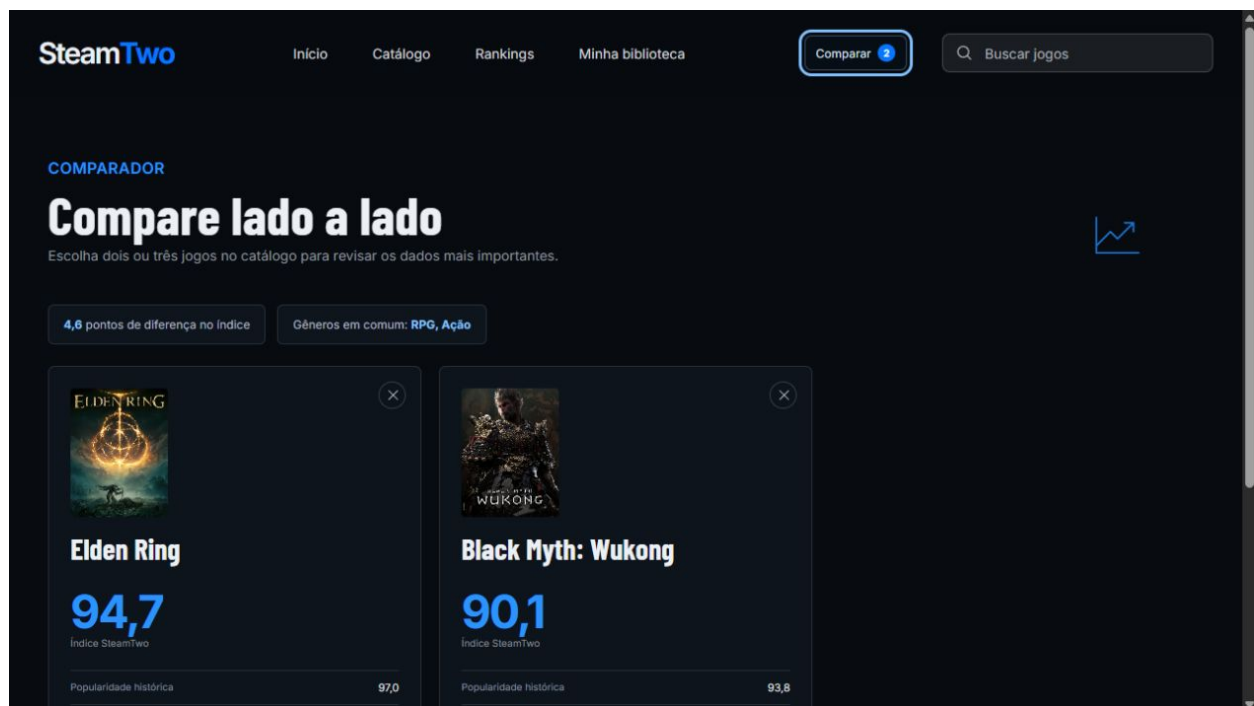


Figura 3 - Comparação entre Elden Ring e Black Myth: Wukong, incluindo diferença de índice e gêneros em comum.

6. Testes, execução e checklist

Verificação	Resultado observado
Testes de domínio, API e integrações	26 testes aprovados em 6 arquivos. Incluem comparação, salvar/atualizar/remover biblioteca e validações.
Build de produção	Concluído com 4.570 módulos transformados.
Teste de preparação para Sites	4 de 4 testes aprovados.
Teste PostgreSQL da entrega	Pendente da credencial da instância do grupo: aplicar migrações, rodar seed e capturar /api/health conectado.

Como executar

1. Clone o novo repositório do grupo e entre na pasta.
2. Instale dependências: `npm install`.
3. Crie `.env` a partir de `.env.example` e informe o `DATABASE_URL` local do grupo.
4. Inicie PostgreSQL (`docker compose up -d`, se usar Docker).
5. Rode `npm run db:migrate` e `npm run db:seed`.
6. Em dois terminais, rode `npm run dev:api` e `npm run dev`.
7. Abra a interface em `http://127.0.0.1:5173/` e a saúde da API em `http://127.0.0.1:3001/api/health`.

Checklist antes de enviar

- ☐ Confirmar o link do repositório na capa após a publicação no GitHub.
- ☐ Executar a integração PostgreSQL do grupo e inserir a captura de `/api/health` conectado.
- ☐ Confirmar `npm test`, `npm run build` e `npm run test:sites` na versão final.
- ☐ Garantir que `.env`, senhas e chaves não foram enviados ao GitHub.